

Tema 1: Introduccion al Lenguaje C y Compilacion

Vamos a empezar con lo mas basico pero importantisimo: entender que es C y como funciona la compilacion. Esto te lo van a preguntar en el examen y es la base de todo lo demas.

Que es C y por que importa en Sistemas Operativos

C es un lenguaje de programacion imperativo y estructurado. Esto quiere decir que las instrucciones se ejecutan una detras de otra, de arriba a abajo, y que usamos estructuras como bucles y condicionales para controlar el flujo del programa. Dentro de los lenguajes de alto nivel, C es uno de los mas cercanos al hardware. Permite acceso directo a memoria mediante punteros, y eso lo convierte en el lenguaje perfecto para programar sistemas operativos.

Una cosa fundamental que debes recordar es que C es un lenguaje de tipado debil. Esto significa que el compilador no se va a quejar si intentas asignar valores de distinto tamaño o signo a una variable. Si asignas un valor demasiado grande a una variable pequena, simplemente se trunca y no te avisa. Los desbordes de variables no se detectan automaticamente. Esto es fuente de muchos errores y preguntas de examen.

Los tipos de datos principales que usamos en esta asignatura son: char, que ocupa un byte; int, que en arquitecturas de 64 bits ocupa 4 bytes; unsigned int, que es un entero sin signo tambien de 4 bytes; long; y void, que se usa para punteros genericos. Es importante que recuerdes estos tamaños porque te van a hacer preguntas de sizeof en el examen.

El proceso de compilacion

La compilacion en C tiene tres fases, y el programa gcc las automatiza todas. La primera fase es el preprocesado: aqui se procesan las directivas que empiezan por almohadilla, como los include, se eliminan comentarios y se sustituyen macros. La segunda fase es la compilacion propiamente dicha, donde se genera codigo objeto, que son archivos con extension punto o. La tercera fase es el enlazado, donde se juntan los ficheros objeto con las bibliotecas necesarias para generar el ejecutable final.

En la practica, los comandos que usamos son: primero, gcc con las opciones menos g, menos c, menos Wall, menos Wshadow y menos Wvla, seguido del nombre del fichero punto c. Esto compila sin enlazar y genera el punto o. La opcion menos g incluye informacion de depuracion para poder usar gdb despues. La opcion menos Wall activa todos los warnings, y es muy importante que la uses siempre porque los warnings son practicamente errores que debes corregir. Despues, para enlazar, usamos gcc menos g menos o seguido del nombre del ejecutable y el fichero punto o. Esto genera el ejecutable final.

Un error muy comun en examenes es confundir las fases. Recuerda: preprocesado, compilacion y enlazado son tres cosas distintas.

Tu primer programa en C

El programa mas basico que puedes escribir es el hola mundo. Empieza con dos includes: `stdlib` punto `h`, que nos da funciones como `exit`, y `stdio` punto `h`, que nos da funciones de entrada y salida como `printf`. Luego viene la funcion `main`, que recibe dos parametros: `argc`, que es un entero con el numero de argumentos, y `argv`, que es un array de cadenas con los argumentos. Dentro, llamamos a `printf` para imprimir texto, y finalmente llamamos a `exit` con `EXIT_SUCCESS` para terminar correctamente el programa.

Hay cosas importantes aqui. Los include deben ir al principio del fichero. Los comentarios en C se escriben entre barra asterisco y asterisco barra, y no se pueden anidar. Todas las sentencias terminan con punto y coma. Un bloque es un grupo de sentencias entre llaves que se tratan como una unidad.

Variables globales y locales

Las variables globales se declaran fuera de cualquier funcion. Son visibles desde cualquier funcion del fichero, se almacenan en el segmento de datos, y si no las inicializas, se ponen a cero automaticamente. Las variables locales se declaran dentro de funciones o bloques. Solo son visibles dentro de ese bloque, se almacenan en la pila o stack, y si no las inicializas, tienen un valor indeterminado, que puede ser cualquier basura que hubiera antes en esa posicion de memoria.

Esto ultimo es muy importante en exámenes: si declaras una variable local sin inicializarla, su valor es impredecible. Las variables globales sin inicializar son cero, pero las locales no.

Existe tambien la palabra clave `static`. Si declaras una variable `static` dentro de una funcion, esa variable conserva su valor entre llamadas a la funcion, porque se almacena en el segmento de datos en vez de en la pila. Esto aparece en algunos ejercicios de examen.

Literales y constantes

En C, los numeros se pueden escribir en distintas bases. Un numero como `777` es decimal. Si empieza por `0x`, como `0x777`, es hexadecimal. Si empieza por cero a secas, como `0777`, es octal. Esto es una trampa clasica de examen: si pones un cero delante de un numero, C lo interpreta como octal.

Las constantes enteras se definen con `enum`, no con `const` ni con `define`. Dentro de un `enum`, los valores se asignan consecutivamente empezando por cero, a menos que especifiques un valor explicitamente.

Operadores

Los operadores aritmeticos son los clasicos: suma, resta, multiplicacion, division y modulo. El modulo solo funciona con enteros.

Los operadores logicos son: doble ampersand para AND, doble barra vertical para OR, y exclamacion para NOT. Recuerda que en C, cero es falso y cualquier otro valor es verdadero.

Las operaciones logicas devuelven cero o uno.

Los operadores de asignacion incluyen el igual simple y las versiones compuestas como mas igual, menos igual, etcetera. Estan tambien los operadores de incremento y decremento: doble mas y doble menos, que pueden ser prefijos o postfijos. Si son prefijos, primero incrementan y luego devuelven el valor. Si son postfijos, primero devuelven el valor y luego incrementan. Esta distincion es pregunta clasica de examen.

Los operadores de bits son: ampersand para AND de bits, barra vertical para OR de bits, acento circunflejo para XOR, virgulilla para NOT de bits o complemento a uno, y los desplazamientos a izquierda y derecha con doble menor y doble mayor. Tambien tenemos sizeof, que devuelve el tamaño en bytes de un tipo o variable. Cuidado con sizeof: si lo aplicas a un puntero, te da el tamaño del puntero, no del bloque de memoria al que apunta. Esto es pregunta estrella de examen.

Control de flujo

La sentencia if ejecuta codigo condicionalmente. Los parentesis son obligatorios. Si solo hay una sentencia, las llaves son opcionales, pero es buena practica ponerlas siempre. La sentencia switch compara un valor contra multiples casos. Si no pones break, la ejecucion cae al siguiente caso, lo que se llama fall-through. Esto es otra trampa de examen.

Los bucles while ejecutan mientras la condicion sea verdadera. El do-while ejecuta al menos una vez y luego comprueba la condicion. El bucle for tiene inicializacion, condicion y actualizacion. Las sentencias break y continue funcionan en todos los bucles: break sale del bucle, y continue salta a la siguiente iteracion.

Un ejemplo tipico de examen es este: un bucle for de cero a cinco donde si i es igual a dos haces continue y si i es mayor que tres haces break. El resultado impreso seria cero, uno, tres. Porque cuando i vale dos, continue salta el printf. Cuando i vale cuatro, break sale del bucle. Y el tres si se imprime porque no cumple ninguna de las dos condiciones.

Funciones

Una funcion es un bloque de codigo reutilizable. La funcion debe estar declarada, es decir, que el compilador conozca su prototipo, antes de usarla. Los argumentos en C siempre son por valor: se pasa una copia. Para modificar una variable desde una funcion, necesitas pasar un puntero a esa variable. Si la funcion no devuelve nada, su tipo es void. Si no tiene argumentos, el prototipo usa void entre parentesis.

Recuerda: la funcion main es el punto de entrada del programa. Recibe argc, que es el numero de argumentos incluyendo el nombre del programa, y argv, que es un array de strings terminado en NULL. Asi que si ejecutas tu programa con dos argumentos, argc vale tres.

Acceso a ayuda: paginas del manual

Las paginas del manual de Unix son esenciales. Puedes acceder a ellas con el comando man seguido del numero de seccion y el nombre. La seccion uno es para comandos del sistema, la

seccion dos para llamadas al sistema como fork o read, y la seccion tres para funciones de biblioteca como printf. El comando apropos te permite buscar por palabra clave.